

Implementation of the -sys and -lib options

RNDr. Karel Šindelka, Ph.D.
k.sindelka@gmail.com

Version 4.x (1st September 2026)

Contents

1	The -sys and -lib options	2
1.1	Where they sit in a utility's start-up	2
1.2	Recording the options	2
1.3	Applying the options	2
1.4	-sys: naming molecule types	3
1.5	-lib: naming bead types	4
1.5.1	What the library holds	4
1.5.2	The top level	4
1.5.3	Pass 1: beads inside named molecules	5
1.5.4	Pass 2: free beads	6
1.5.5	Summary of the warnings	8
1.6	Why the order matters	8
1.7	Utilities that do more	9

1. The `-sys` and `-lib` options

The two common options change what a system's types are *called* rather than what is in it: `-sys <file>` names the molecule types and `-lib <dir>` names the bead types (and gives them their masses, charges, and radii). These options make sense of a file that carries coordinates and topology but no useful names, typically a `data` file produced by LAMMPS.

The options are not independent: the library matches molecules *by name*, so on a nameless input, `-lib` can do its job only if `-sys` has run first. That ordering is fixed internally, the options' order in the command does not matter.

TODO: Only `-sys` and only `-lib` comment?

1.1 Where they sit in a utility's start-up

Every utility follows the following three steps (in that order):

- 1) `CommonOptions()` records the file and directory names
- 2) `ReadStructure()` builds the `SYSTEM` described by the input file
- 3) `ApplyLibraryOptions()` applies `-sys`, then `-lib`

Step 3 must come before any option that names a type (e.g., `-bt`, `-mt`, or a molecule-name argument), so that those options can use the library's names rather than the input file's.

1.2 Recording the options

`CommonOptions()` stores both file and directory names in the `COMMON_OPT` struct. An empty string marks 'unused option' throughout.

```
1: opt.lib ← ""
2: FileOption(-lib, opt.lib)           ▷ fills it in only if -lib is present
3: opt.sys ← ""
4: FileOption(-sys, opt.sys)
```

1.3 Applying the options

void ApplyLibraryOptions (parameters)			ReadLibrary.c
parameter	in/out	explanation	
(COMMON_OPT) commons	in	holds the <code>-sys</code> and <code>-lib</code> file and dir names	
(SYSTEM) sys	in/out	the system whose types get renamed	

(LIBRARY) lib	out	the library dir; used only when <code>free_library</code> is <code>false</code>
(bool) free_library	in	whether the library is freed here (<code>true</code> for all but <code>Info</code>)

```

1: if commons.sys ≠ "" then
2:   ReadSysInfo(commons.sys, sys)
3: end if
4: if commons.lib ≠ "" then
5:   ApplyLibraryToSystem(commons.lib, sys, lib, free_library)
   ▷ -sys has already run
6: end if

```

`free_library` decides who owns the library once the renaming is done, and is the only thing separating the two kinds of caller:

- `true`: the library is read, used and freed inside the call, and `lib` is ignored (case for most utilities)
- `false`: the library is handed back in `lib`, so the caller must use `FreeLibrary()` to free it

Only `Info` passes `false`, because it needs the library to print the DPD interaction parameters.

1.4 -sys: naming molecule types

`Info` writes the `-sys` file with `WriteSysInfo()`: one `<mol name> <count>` line per molecule type. Reading it back is purely *positional*: the i -th valid line names the i -th molecule type, meaning the molecule types to rename in the input structure file must be in the same order. The `<count>` is parsed only to decide whether a line is a valid entry; nothing cross-checks it against the system.

void ReadSysInfo(parameters)		ReadWriteSysInfo.c
parameter	in/out	explanation
(char) filename[]	in	the <code>-sys</code> file
(SYSTEM) System	in/out	molecule types get <code>Name</code> and <code>Named</code>

```

1: open filename
2: i ← 0                                     ▷ index of the molecule type being
                                           named
3: while line ← next line of filename do
4:   if line is empty, starts with #, has fewer than 2 words, or 2nd word of line
     is not a whole number then
5:     continue                             ▷ skip blank, comment, and invalid
                                           lines
6:   end if
7:   if i ≥ System.Count.MoleculeType then
8:     warn: more entries than molecule types; only the first i are used

```

```

9:      break
10:    end if
11:    System.MoleculeType[i].Name ← 1st word of line
12:    System.MoleculeType[i].Named ← true
13:    i ← i + 1
14: end while
15: close filename
16: if i < System.Count.MoleculeType then
17:   warn: i entries for a system with Count.MoleculeType molecule types
18: end if

```

Two notes:

- because the match is positional, a file whose lines are in the wrong order will misname every type (both warnings above are about *counts of entries*, not about whether the names make sense)
- `-sys` touches no bead types; on a nameless `data` file, it enables `-lib` to work (section 1.5.3), but on its own (no `-lib` option) it changes only the molecule names

1.5 -lib: naming bead types

1.5.1 What the library holds

`ReadLibrary()` reads four `list_*.txt` tables from the library directory into a `LIBRARY`:

file	what it contributes
<code>list_beadtypes.txt</code>	bead types (name, mass, charge, radius) and their self-interactions
<code>list_bonds.txt</code>	bond types, each with a string ID
<code>list_angles.txt</code>	angle types, each with a string ID
<code>list_cross_interactions.txt</code>	interactions between unlike bead types

Everything else in the directory is one molecule per `<mol name>.txt` file, read on demand by `ReadLibraryMolFile()`. Those files are what the renaming below keys on; the tables only supply the bead data that gets copied across.

1.5.2 The top level

<code>void ApplyLibraryToSystem(parameters)</code>			<code>ReadLibrary.c</code>
parameter	in/out	explanation	
<code>(char) lib_dir[]</code>	in	the <code>-lib</code> directory	
<code>(SYSTEM) sys</code>	in/out	the system whose bead types get renamed	

(LIBRARY) <code>lib</code>	out	the library, for the caller to <code>FreeLibrary()</code> ; ignored when <code>free_library</code>
(bool) <code>free_library</code>	in	whether the library is freed here

```

1: lib ← lib, or a local variable if free_library a null lib is an error otherwise
2: before ← copy of sys.BeadType[]                ▷ to tell afterwards whether
                                                    anything happened
3: lib ← ReadLibrary(lib_dir)
4: RenameBeadTypesFromLibrary(sys, lib, lib_dir)
5: changed ← true if any bead type's name, mass, charge or radius differs from
   before
6: unmatched ← number of molecule types in sys with no <Name>.txt in lib_dir
7: if not changed then
8:   warn: no bead type matched the library
9: else if unmatched > 0 then
10:  warn: unmatched molecule type(s) have no library file, so the beads inside
    them keep what the input gave them; name them with -sys if the input does
    not
11: end if
12: if free_library then
13:   FreeLibrary(lib)
14: end if

```

A library that changes nothing is almost certainly the wrong directory, or an input whose molecule types are unnamed, so it is worth a warning rather than carrying on quietly with the file's own (possibly missing) masses.

`RenameBeadTypesFromLibrary()` then does the real work in two passes: bead types used inside molecules are handled by molecule name, and free (unbonded) bead types by charge. The two sets do not overlap, so neither pass can undo the other.

1.5.3 Pass 1: beads inside named molecules

A molecule type whose name matches a library file lends its bead names, position by position. The catch is that one *system* bead type can be reached from several *library* beads at once — a `data` file with no `Masses` section has nothing to tell its neutral beads apart, so they all arrive as a single type. Renaming as we go would give such a type whichever library name came last, so proposals are collected first and applied only once every molecule has been seen.

```

1: for each bead type i of sys do
2:   proposed[i] ← none
3: end for
4: for each molecule type mt of sys do
5:   if mt.Name is empty then
6:     continue
7:   end if
8:   if the file named mt.Name.txt cannot be read from lib_dir then

```

```

9:      continue
10:    end if
11:     $mol \leftarrow$  that file
12:    if  $mol.n\_beads \neq mt.nBeads$  then
13:      continue ▷ not the molecule the name
suggests; counterions are separate
molecules, so the counts must
agree
14:    end if
15:    for each bead position  $b$  of  $mol$  do
16:       $st \leftarrow mt.Bead[b]$  ▷ system bead type at that position
17:       $lt \leftarrow FindBeadType(mol.bead\_name[b], lib)$  ▷ by name
18:      if  $lt$  is none then
19:        continue
20:      end if
21:      if  $proposed[st]$  is none then
22:         $proposed[st] \leftarrow lt$ 
23:      else if  $proposed[st] \neq lt$  then
24:         $proposed[st] \leftarrow ambiguous$ 
25:      end if
26:    end for
27:  end for
28:   $ambiguous \leftarrow 0$ 
29:  for each bead type  $i$  of sys do
30:    if  $proposed[i]$  is ambiguous then
31:       $ambiguous \leftarrow ambiguous + 1$ 
32:      continue ▷ left exactly as the input had it
33:    end if
34:    if  $proposed[i]$  is none then
35:      continue
36:    end if
37:    copy Name, Charge, Mass and Radius from library bead type  $proposed[i]$ 
38:  end for
39:  if  $ambiguous > 0$  then
40:    warn:  $ambiguous$  bead type(s) stand for several library beads at once and
      are left alone; the input cannot tell them apart
41:  end if

```

The guiding rule, here and in pass 2: *a wrong name is worse than no name.*

1.5.4 Pass 2: free beads

Beads that belong to no molecule cannot be reached by name, because there is no molecule name to look up. They are matched by charge instead, and by count where the count is predictable. Candidates come from walking the library directory:

```

1: for each bead type  $bt$  used by any molecule type of sys do
2:    $in\_mol[bt] \leftarrow true$  ▷ pass 1's territory; skipped below

```

```

3: end for
4: cand  $\leftarrow$  empty list
5: for each *.txt in lib_dir that is not a list_* table do
6:   mol  $\leftarrow$  that file, skipping it if unreadable
7:   if mol has exactly one bead, known to the library then
8:     append {name, charge, count = unknown} to cand
     $\triangleright$  any single-bead molecule is loose
    in the system, salts included
9:   end if
10:  parent  $\leftarrow$  number of molecules in sys whose type is named mol.name
11:  if parent = 0 then
12:    continue  $\triangleright$  its counterions are not in this system
13:  end if
14:  for each counterion c declared by mol do
15:    cion  $\leftarrow$  the file named c.name.txt, skipping it if unreadable or not monoatomic
16:    find or append the cand entry for c.name, with count starting at 0
17:    count  $\leftarrow$  count + parent  $\times$  c.count  $\triangleright$  summed over every parent molecule
18:  end for
19: end for

```

Matching then goes from the most reliable evidence to the least. A name the library already knows beats any guess made from charge — the file has said what the type is, so there is nothing to work out:

```

1: for each bead type bt with in_mol[bt] false do
2:   lt  $\leftarrow$  FindBeadType(sys.BeadType[bt].Name, lib)
3:   if lt is none then
4:     continue
5:   end if
6:   copy Charge, Mass and Radius from library bead type lt
     $\triangleright$  the name already is the library's
7:   in_mol[bt]  $\leftarrow$  true  $\triangleright$  done with; the charge matching below skips it
8: end for
9: for each bead type bt with in_mol[bt] still false do
10:  q  $\leftarrow$  sys.BeadType[bt].Charge, n  $\leftarrow$  sys.BeadType[bt].Number
11:  match  $\leftarrow$  the only cand with a known count,  $|charge - q| < 0.01$  and count = n
12:  if match is none then
13:    match  $\leftarrow$  the only cand with an unknown count and  $|charge - q| < 0.01$ 
14:  end if
15:  if match is not none then
16:    Name  $\leftarrow$  match.name; copy Charge, Mass and Radius from the library
17:  end if
18: end for

```

Both ‘the only’ steps rename nothing when two or more candidates fit. Note that an ambiguous charge-plus-count match does not give up: it falls through to the charge-only step, which may still find a unique answer among the candidates whose count is unpredictable.

Leaving single-bead molecules that declare counterions of their own out of the candidate list is what once made an **Na+** bead get renamed to the identically charged **H+**; they are included precisely because they have no predictable count and can only be matched on charge.

1.5.5 Summary of the warnings

warning	what it means
no bead type matched the library	nothing at all changed: wrong directory, or an input whose molecule types are unnamed and whose free beads match nothing
<n> molecule type(s) have no library file	those molecules’ beads keep what the input gave them — add -sys if the input does not name its molecules
<n> bead type(s) stand for several library beads	pass 1 found conflicting proposals; the input cannot tell those beads apart, so they are left alone

1.6 Why the order matters

Examples/SystemID/01-FilesAndOptions builds the same system in several file formats and then strips information out of them, which makes the division of labour easy to see. Starting from **foreign.data** — a **data** file with no name comments, so bead types arrive as **b0 ... b4** and the molecule type as **m**:

options	result
-sys alone	molecule types get their names; every bead type is untouched
-lib alone	pass 1 finds no molecule file to match, so only the free beads are identified (water by charge, the counterion by charge and count); warns
-lib and -sys	-sys runs first, so pass 1 can match the molecule and every bead type is identified

The refusal case is worth seeing too: on **nomass.data** (the same file with the **Masses** section removed) even **-lib** with **-sys** leaves the molecule’s neutral beads alone, because they arrive as one bead type standing for three library beads — pass 1’s *ambiguous* outcome.

1.7 Utilities that do more

- `Info` declares its own `-lib` and `-sys` help entries and is the one caller passing `free_library` as `false`, to print the DPD interaction parameters and to write a FIELD file's interactions. It also has `--check-library` and `--mol-info`, which need only `-lib` and no input file at all.
- `AddToSystem` uses `-lib` as a *source of molecules* to add via `-mol`, which is a different job from renaming what is already there. Its `-sys` takes an optional second file name to write the updated composition to, and when building a system from scratch a single argument is treated as output rather than input.